

Discrete Fourier Transform and Spatial Convolution Filtering Using OpenCV

Aufa Rienaldifaza Ahmad - 13524027

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Bandung, Indonesia

aufa.rienaldifaza.a@gmail.com

I. THEORETICAL FOUNDATION

A. Linear Spatial Filtering

Convolution combines every part of an image with a kernel: a fixed size array of coefficients with an anchor point at its center [1]. The kernel is rotated 180° about its anchor, then slid over the image; at each position its coefficients are multiplied with the underlying pixels, summed, and written back at the anchor. For an image I and kernel K of size $M_i \times M_j$ with anchor (a_i, a_j) ,

$$H(x, y) = \sum_{i=0}^{M_i-1} \sum_{j=0}^{M_j-1} K(i, j) I(x - i + a_i, y - j + a_j) \quad (1)$$

OpenCV's `cv::filter2D` instead computes *correlation* (the same sum without the 180° rotation), so a kernel must be pre-rotated to obtain true convolution through it; kernels symmetric under 180° rotation give identical results either way.

B. Discrete Fourier Transform

The Fourier Transform decomposes an image into sine and cosine components in the frequency domain [2]. For an $N \times N$ image $f(i, j)$,

$$F(k, l) = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} f(i, j) e^{-i2\pi(\frac{ki}{N} + \frac{lj}{N})} \quad (2)$$

which is complex-valued even for real-valued input. Since `cv::Mat` has no native complex type, the real and (zero) imaginary planes are interleaved with `cv::merge` before the transform and separated with `cv::split` after. DFT performance is best for sizes that are products of small primes, so the input is zero-padded to `cv::getOptimalDFTSize` first.

A complex value cannot be shown as a pixel, so the spectrum is visualized through its magnitude,

$$M(k, l) = \sqrt{\text{Re}(F(k, l))^2 + \text{Im}(F(k, l))^2} \quad (3)$$

The DC term dominates M by orders of magnitude, so a logarithmic transform $M_1 = \log(1 + M)$ is applied before display. The raw DFT also places the zero-frequency term at the corner $(0, 0)$; swapping diagonally opposite quadrants recenters it for interpretability.

II. EXPERIMENTAL RESULTS

A. Implementation

The methods described in Section I-A and Section I-B were implemented as member functions of an `ImageProcessor` class, built as an OpenCV-based `ament_cmake` library. Three filtering-related methods were implemented:

- `convolutionFilter`: applies a user-supplied kernel using OpenCV's `cv::filter2D`, which performs correlation rather than true convolution (Section I-A), so the kernel is first rotated 180° with `cv::rotate` so that the combined operation matches Eq. 1.
- `convolutionFilterManual`: a from-scratch implementation of the same mathematical operation. The kernel is applied as a convolution directly, by indexing it in reverse order rather than pre-rotating it, and out-of-bounds neighbors are treated as zero, matching `convolutionFilter`'s border behavior.
- `fourierTransform`: computes the log-magnitude spectrum of an image following the pipeline in Section I-B (optimal-size padding, complex plane construction, DFT, magnitude, log scaling, and optional quadrant centering).

All three methods were exercised through a demo executable on the 512×512 , 3-channel `Lenna.png` test image, shown unmodified in Fig. 1.



Fig. 1. Original 512×512 test image (Lenna.png).

B. Correctness of the Manual Convolution

`convolutionFilterManual` was validated against `convolutionFilter` using a 15×15 Gaussian kernel from `cv::getGaussianKernel`. Results are shown in Fig. 2 and Fig. 3.



Fig. 2. Result of `convolutionFilter` (filter2D-based) using the 15×15 Gaussian kernel K .



Fig. 3. Result of `convolutionFilterManual` using the same kernel K , visually indistinguishable from Fig. 2.

C. Performance Comparison

TABLE I
AVERAGE RUNTIME PER FRAME (512×512×3 IMAGE, 3×3 KERNEL)

Method	Time (ms/frame)
<code>convolutionFilter</code> (filter2D)	0.27
<code>convolutionFilterManual</code> (loops)	11.96

The manual implementation is approximately $45\times$ slower than OpenCV's `filter2D`. This is due to `filter2D` being backed by vectorized (SIMD), cache-optimized, and potentially multi-threaded code, whereas the manual version performs one scalar multiply-accumulate per kernel tap through plain nested loops.

D. Fourier Spectrum

The log-magnitude spectrum computed by `fourierTransform` shows a bright peak at the center corresponding to the zero-frequency (DC) component, surrounded by a star-shaped pattern of radial spikes aligned with the image's row and column edges after border padding. Disabling the quadrant centering step reproduces the same spectrum with the DC term located at the corners instead, consistent with the raw output layout of `cv::dft`.

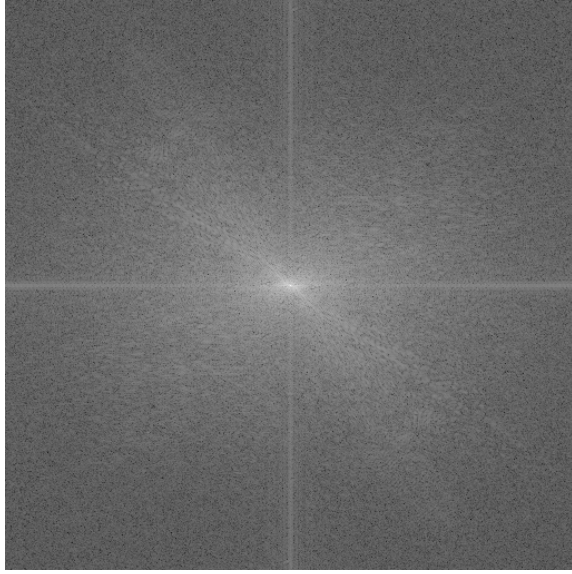


Fig. 4. Log-magnitude Fourier spectrum of `Lenna.png`, quadrant-centered so the zero-frequency term appears at the middle of the image.

E. Effect of Kernel Size on Blur Strength

Initial testing with a small 3×3 box-averaging kernel $K = \frac{1}{9}\mathbf{1}_{3 \times 3}$ produced only a mild blur. Box kernels $K_N = \frac{1}{N^2}\mathbf{1}_{N \times N}$ were therefore applied to `Lenna.png` via `convolutionFilter` for $N \in \{3, 9, 15, 21, 31\}$, shown in Fig. 5. Blur strength increases visibly and steadily with N , becoming clearly soft from around $N = 15$ onward.

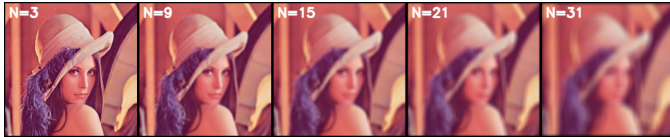


Fig. 5. Box-blur result on `Lenna.png` for increasing kernel size N , using `convolutionFilter`.

APPENDIX

The full project source is available at https://github.com/aufafaza/EL4007-Digital-Image-Processing_project. The five ImageProcessor methods are reproduced below for reference.

toGrayscale

```
cv::Mat ImageProcessor::toGrayscale(const cv::Mat & input) const
{
    if (input.empty()) {
        throw std::invalid_argument("ImageProcessor::toGrayscale:_input_image_is_empty");
    }
    if (input.channels() == 1) {
        return input.clone();
    }
    cv::Mat gray;
    cv::cvtColor(input, gray, cv::COLOR_BGR2GRAY);
    return gray;
}
```

gaussianBlur

```
cv::Mat ImageProcessor::gaussianBlur(const cv::Mat & input, int kernel_size) const
{
    if (input.empty()) {
        throw std::invalid_argument("ImageProcessor::gaussianBlur:_input_image_is_empty");
    }
    if (kernel_size % 2 == 0) {
        kernel_size += 1;
    }
    cv::Mat blurred;
    cv::GaussianBlur(input, blurred, cv::Size(kernel_size, kernel_size), 0);
    return blurred;
}
```

convolutionFilter

```
cv::Mat ImageProcessor::convolutionFilter(const cv::Mat & input) const
{
    if (input.empty()) {
        throw std::invalid_argument("ImageProcessor::convolutionFilter:_input_image_is_empty");
    }
    if (this->kernel_.empty()) {
        throw std::runtime_error("kernel_is_empty");
    }
    cv::Mat filtered;

    // rotate kernel 180 degrees so filter2D's correlation becomes convolution
    cv::rotate(kernel_, kernel_, cv::ROTATE_180);

    cv::filter2D(input, filtered, -1, kernel_, cv::Point(-1, -1), 0, cv::BORDER_CONSTANT);

    return filtered;
}
```

convolutionFilterManual

```
cv::Mat ImageProcessor::convolutionFilterManual(const cv::Mat & input) const
{
    if (input.empty()) {
        throw std::invalid_argument("ImageProcessor::convolutionFilterManual:_input_image_is_empty");
    }
    if (kernel_.empty()) {
        throw std::runtime_error("kernel_is_empty");
    }
    if (input.depth() != CV_8U) {
        throw std::invalid_argument(
            "ImageProcessor::convolutionFilterManual:_only_8-bit_images_are_supported");
    }

    cv::Mat kernel_f;
    kernel_.convertTo(kernel_f, CV_32F);

    const int kh = kernel_f.rows;
    const int kw = kernel_f.cols;
    const int anchor_y = kh / 2;
    const int anchor_x = kw / 2;

    const int rows = input.rows;
    const int cols = input.cols;
    const int channels = input.channels();
}
```

```

cv::Mat output = cv::Mat::zeros(input.size(), input.type());

for (int y = 0; y < rows; ++y) {
    uchar * out_row = output.ptr<uchar>(y);

    for (int x = 0; x < cols; ++x) {
        for (int c = 0; c < channels; ++c) {
            float sum = 0.0f;

            for (int i = 0; i < kh; ++i) {
                int sy = y + i - anchor_y;
                if (sy < 0 || sy >= rows) {
                    continue; // zero padding
                }
                const uchar * in_row = input.ptr<uchar>(sy);

                for (int j = 0; j < kw; ++j) {
                    int sx = x + j - anchor_x;
                    if (sx < 0 || sx >= cols) {
                        continue; // zero padding
                    }
                    float weight = kernel_f.at<float>(kh - 1 - i, kw - 1 - j);
                    sum += weight * static_cast<float>(in_row[sx * channels + c]);
                }
            }

            out_row[x * channels + c] = cv::saturate_cast<uchar>(sum);
        }
    }
}

return output;
}

```

fourierTransform

```

cv::Mat ImageProcessor::fourierTransform(const cv::Mat & input, bool shift) const
{
    if (input.empty()) {
        throw std::invalid_argument("ImageProcessor::fourierTransform:_input_image_is_empty");
    }

    cv::Mat gray = toGrayscale(input);

    int opt_rows = cv::getOptimalDFTSize(gray.rows);
    int opt_cols = cv::getOptimalDFTSize(gray.cols);
    cv::Mat padded;
    cv::copyMakeBorder(
        gray, padded, 0, opt_rows - gray.rows, 0, opt_cols - gray.cols,
        cv::BORDER_CONSTANT, cv::Scalar::all(0));

    cv::Mat planes[] = {cv::Mat_<float>(padded), cv::Mat::zeros(padded.size(), CV_32F)};
    cv::Mat complex_img;
    cv::merge(planes, 2, complex_img);

    cv::dft(complex_img, complex_img);

    cv::split(complex_img, planes);
    cv::magnitude(planes[0], planes[1], planes[0]);
    cv::Mat magnitude = planes[0];

    magnitude += cv::Scalar::all(1);
    cv::log(magnitude, magnitude);

    magnitude = magnitude(cv::Rect(0, 0, magnitude.cols & -2, magnitude.rows & -2));

    if (shift) {
        int cx = magnitude.cols / 2;
        int cy = magnitude.rows / 2;

        cv::Mat q0(magnitude, cv::Rect(0, 0, cx, cy));
        cv::Mat q1(magnitude, cv::Rect(cx, 0, cx, cy));
        cv::Mat q2(magnitude, cv::Rect(0, cy, cx, cy));
        cv::Mat q3(magnitude, cv::Rect(cx, cy, cx, cy));

        cv::Mat tmp;
        q0.copyTo(tmp);
        q3.copyTo(q0);
        tmp.copyTo(q3);

        q1.copyTo(tmp);
        q2.copyTo(q1);
        tmp.copyTo(q2);
    }
}

```

```
}  
  
cv::normalize(magnitude, magnitude, 0, 255, cv::NORM_MINMAX);  
magnitude.convertTo(magnitude, CV_8U);  
  
return magnitude;  
}
```

REFERENCES

- [1] OpenCV Development Team, “Making your own linear filters!,” *OpenCV Tutorials*. [Online]. Available: https://docs.opencv.org/4.x/d4/dbd/tutorial_filter_2d.html
- [2] OpenCV Development Team, “Discrete Fourier Transform,” *OpenCV Tutorials*. [Online]. Available: https://docs.opencv.org/4.x/d8/d01/tutorial_discrete_fourier_transform.html